

```
// OSSP Skill 10 - pwd, exit, and export
// Demonstrates current directory retrieval, exit handling,
// resource cleanup, export syntax, environment update,
// and inheritance by a child process.

#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <sys/wait.h>
#include <limits.h>
#include <ctype.h>

#ifndef PATH_MAX
#define PATH_MAX 4096
#endif

static void cleanup(void) {
    printf("Cleaning up shell resources...\n");
}

static int valid_name(const char *name) {
    if (!name || (!isalpha((unsigned char)name[0]) && name[0] != '_'))
        return 0;

    for (int i = 1; name[i]; i++)
        if (!isalnum((unsigned char)name[i]) && name[i] != '_')
            return 0;
}
```

```
    return 1;
}

static void do_export(char *text) {
    char *eq = strchr(text, '=');

    if (!eq) {
        printf("Usage: export NAME=value\n");
        return;
    }

    *eq = '\0';
    const char *name = text;
    const char *value = eq + 1;

    if (!valid_name(name)) {
        printf("Invalid environment variable name.\n");
        return;
    }

    if (setenv(name, value, 1) == -1)
        perror("setenv");
    else
        printf("Exported %s=%s\n", name, value);
}
```

```
int main(void) {
    char line[256];

    atexit(cleanup);

    printf("=== Skill 10: pwd / export / exit ===\n");

    while (1) {
        char cwd[PATH_MAX];

        printf("envshell> ");
        if (!fgets(line, sizeof(line), stdin))
            break;

        line[strcspn(line, "\n")] = '\0';

        if (strcmp(line, "pwd") == 0) {
            if (getcwd(cwd, sizeof(cwd)))
                printf("%s\n", cwd);
            else
                perror("getcwd");
        } else if (strncmp(line, "export ", 7) == 0) {
            do_export(line + 7);
        } else if (strcmp(line, "show") == 0) {
            const char *demo = getenv("DEMO");
            printf("DEMO=%s\n", demo ? demo : "<undefined>");
        } else if (strcmp(line, "child") == 0) {
            pid_t pid = fork();

            if (pid == -1) {
                perror("fork");
            } else if (pid == 0) {
                execlp("sh", "sh", "-c", "echo Child sees DEMO=$DEMO", NULL);
                perror("execlp");
                _exit(1);
            } else {
                waitpid(pid, NULL, 0);
            }
        } else if (strcmp(line, "exit") == 0) {
            printf("Exit requested.\n");
            break;
        } else if (line[0] != '\0') {
            printf("Unknown command.\n");
        }
    }

    return 0;
}
```

```
sai_pavani@Sai-Pavani:~/OS_SKILL$ nano skill_10.c.c
sai_pavani@Sai-Pavani:~/OS_SKILL$ gcc skill_10.c.c -o skill_10
sai_pavani@Sai-Pavani:~/OS_SKILL$ ./skill_10
=== Skill 10: pwd / export / exit ===
envshell> pwd
/home/sai_pavani/OS_SKILL
envshell> export DEMO=HelloOSSP
Exported DEMO=HelloOSSP
envshell> show
DEMO=HelloOSSP
envshell> child
Child sees DEMO=HelloOSSP
envshell> exit
Exit requested.
Cleaning up shell resources...
```